AhamX Bodhi

Where I Becomes Infinite

Projection Layers

Module 3.4: Language Model Head and Training Targets

Prof. Sashikumaar Ganesan | IISc Bengaluru

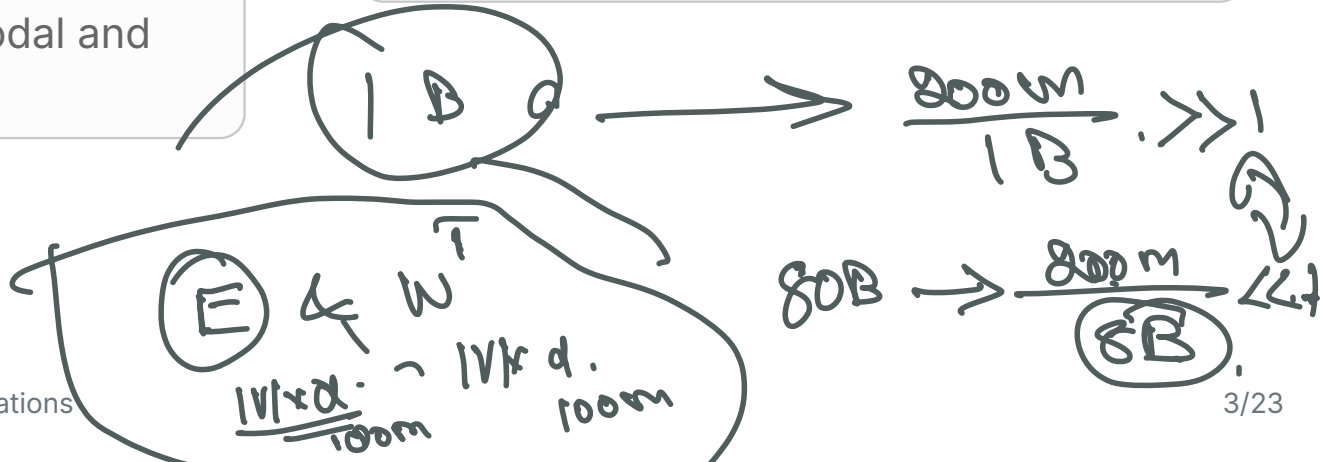
Session Overview

What You Will Learn

- What a projection layer is and why it is needed
- Q, K, V projections in attention
- The output projection $\mathbf{W}^O$ that merges attention heads
- Up/down projections in the feed-forward network (FFN)
- Projection layers in multimodal and adapter architectures

Concept at a Glance

- **Duration:** 8 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Domain:** Theory
- **Key Idea:** Linear maps between representation spaces



Learning Objectives

By the End of This Concept You Will Be Able To

- **Define** a projection layer as a linear map between two vector spaces
- **Identify** all projection matrices in one Transformer block: Q, K, V, O, FFN up/down
- **Explain** why attention projections split and recombine the model dimension
- **Describe** how projection layers enable low-rank adaptation (LoRA) in fine-tuning
- **Connect** projection layers to multimodal alignment (vision-language models)


FFN

What is a Projection Layer?

Core Definition

- A **projection layer** is a single linear (affine) transformation: $\mathbf{y} = \mathbf{x}\mathbf{W} + \mathbf{b}$
- It maps a vector from one dimension d_{in} to another d_{out}
- No activation function: the operation is **purely linear**
- In Transformers, **all projections are weight matrices** $\mathbf{W}$, usually without bias $\mathbf{b}$
- The entire Transformer block is built from: projection layers + layer norms + a softmax

Despite their simplicity, projection layers are the most numerous computationally intensive components in a Transformer. In Llama 3 8B, they account for over 90% of the total parameter count and FLOP cost.

All Projections in One Transformer Block

An Inventory of Every Projection Matrix

- **Attention Query:** $\mathbf{W}^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ per head — maps input to query space
- **Attention Key:** $\mathbf{W}^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ per head — maps input to key space
- **Attention Value:** $\mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ per head — maps input to value space
- **Attention Output:** $\mathbf{W}^O \in \mathbb{R}^{h \cdot d_v \times d_{\text{model}}}$ — merges all heads back to model dimension
- **FFN Up:** $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ — expands to wide intermediate space
- **FFN Down:** $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ — compresses back to model dimension

Query, Key, Value Projections

How Input is Split for Multi-Head Attention

- For each attention head i, the same input $\mathbf{X}$ is projected three ways:

$$\mathbf{Q}_i = \mathbf{X}\mathbf{W}_i^Q, \quad \mathbf{K}_i = \mathbf{X}\mathbf{W}_i^K, \quad \mathbf{V}_i = \mathbf{X}\mathbf{W}_i^V$$

- Each head sees the input from a different "viewpoint" determined by its projection weights
- For multi-head attention with h heads: $d_k = d_v = d_{\text{model}}/h$
- Example: GPT-2 small — $d_{\text{model}} = 768$, $h = 12$, $d_k = 64$ per head
- The projections are the mechanism by which the model learns **what to attend to**

The Output Projection $\mathbf{W}^O$

Merging All Heads into One Representation

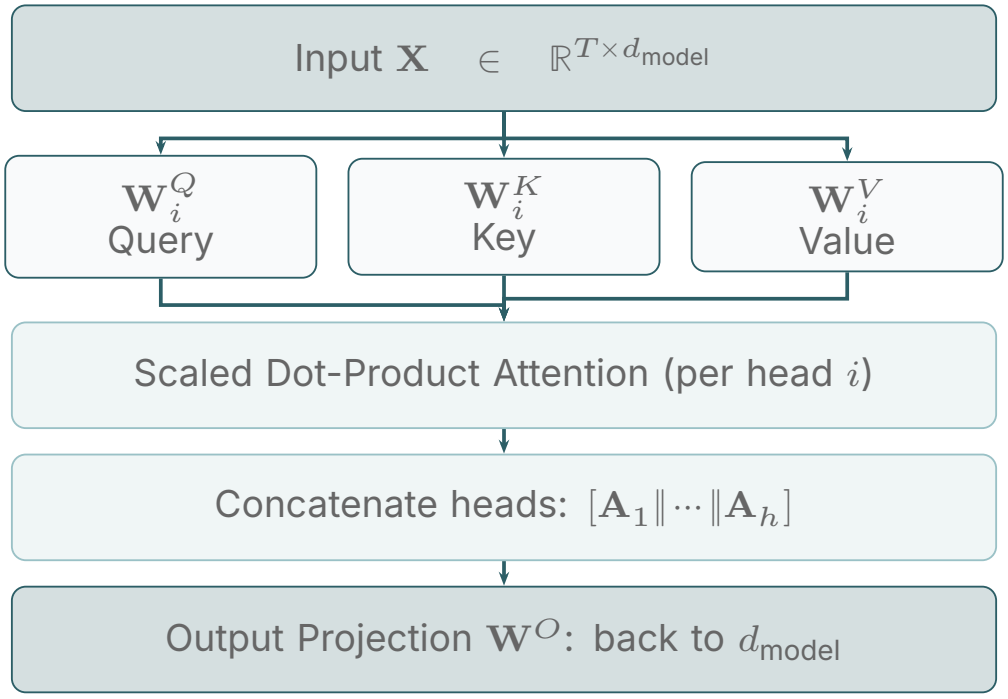
- After each head computes its attention output $\mathbf{A}_i \in \mathbb{R}^{T \times d_v}$, all heads are concatenated:

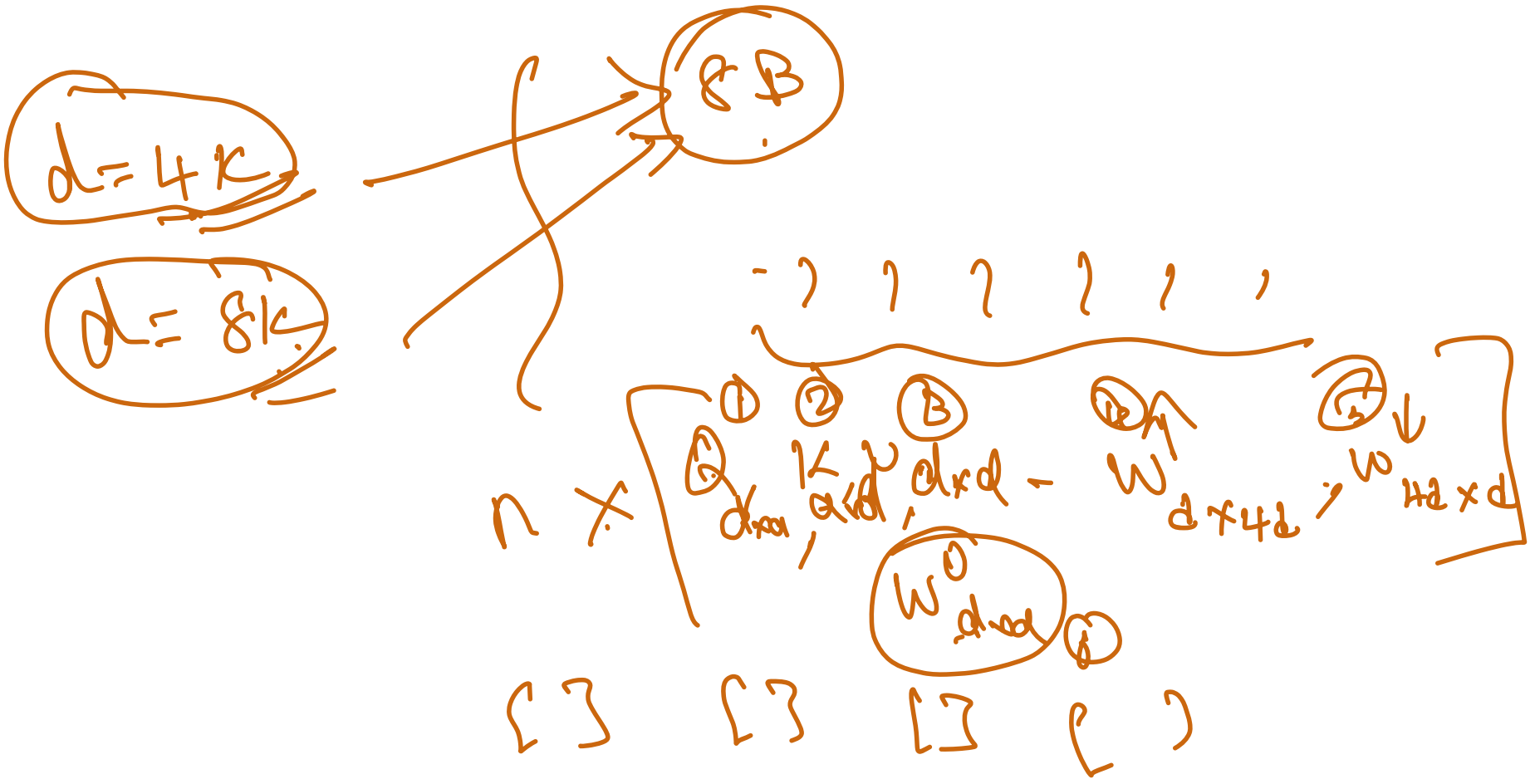
$$\text{MultiHead}(\mathbf{X}) = [\mathbf{A}_1 \parallel \mathbf{A}_2 \parallel \cdots \parallel \mathbf{A}_h]\,\mathbf{W}^O$$

- $\mathbf{W}^O \in \mathbb{R}^{h \cdot d_v \times d_{\text{model}}}$ reduces the concatenated output back to d_{model}
- This is the **only projection that mixes information across heads**
- Without $\mathbf{W}^O$, heads would be independent and their outputs could not interact

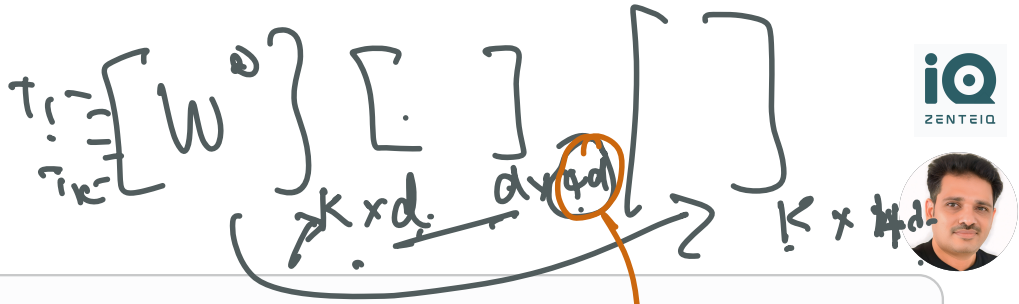
The output projection is sometimes called the **"down-projection"** of the attention block, analogous to the FFN down-projection. Both serve the same structural role: collapsing an expanded representation back into d_{model}.

Attention Projections: Visual Summary




d = 4K
d = 8K
8B

FFN Up and Down Projections



The Expand-and-Compress Pattern

- The feed-forward sublayer applies two projections with a non-linearity between them:

$$\text{FFN}(\mathbf{x}) = \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\,\mathbf{W}_2 + \mathbf{b}_2$$

- $\mathbf{W}_1$ (up-projection): expands from d_{model} to $d_{\text{ff}} = 4 \times d_{\text{model}}$
- $\mathbf{W}_2$ (down-projection): compresses from d_{ff} back to d_{model}
- The activation σ (ReLU, GELU, or SwiGLU) introduces the necessary non-linearity
- The intermediate dimension $4 \times d_{\text{model}}$ is a design choice; Llama uses $\approx 2.67 \times d_{\text{model}}$ with SwiGLU

The FFN is sometimes described as a **"key-value memory"**: the up-projection retrieves patterns; the down-projection writes them back. Research (Geva et al., 2021) suggests FFN layers store factual knowledge that can be edited post-training.

SwiGLU: Llama's FFN Projection Variant

Standard FFN (ReLU/GELU)

- Two matrices: $\mathbf{W}_1$ (up), $\mathbf{W}_2$ (down)
- $\text{FFN}(\mathbf{x}) = \text{GELU}(\mathbf{x}\mathbf{W}_1)\mathbf{W}_2$
- Used in original Transformer, BERT, GPT-2

SwiGLU (Llama, PaLM, Gemma)

- **Three** matrices: $\mathbf{W}_{\text{gate}}$, $\mathbf{W}_{\text{up}}$, $\mathbf{W}_{\text{down}}$
- $\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W}_{\text{gate}} \odot \text{SiLU}(\mathbf{x}\mathbf{W}_{\text{up}}))\,\mathbf{W}_{\text{down}}$
- Gated activation improves training stability and model quality
- 25–30% fewer parameters for same effective capacity

Projection Dimensions: Reference Table

Model	d_{model}	**Heads** h	d_k	d_{ff}	**FFN Type**
GPT-2 Small	768	12	64	3072	GELU
GPT-2 Large	1280	20	64	5120	GELU
Llama 2 7B	4096	32	128	11008	SwiGLU
Llama 3 8B	4096	32	128	14336	SwiGLU
Mistral 7B	4096	32	128	14336	SwiGLU

GenAI Application: LoRA --- Low-Rank Projection Adaptation

Fine-Tuning by Adding Projection Pairs

- LoRA (Hu et al., 2021) decomposes the weight update $\Delta\mathbf{W} = \mathbf{AB}$ where $\mathbf{A} \in \mathbb{R}^{d\times r}$, $\mathbf{B} \in \mathbb{R}^{r\times d}$, rank $r \ll d$
- Instead of updating $\mathbf{W}$ (millions of parameters), only $\mathbf{A}$ and $\mathbf{B}$ are trained (thousands)
- LoRA is applied to the Q, K, V, and O projection matrices in attention
- At inference, $\mathbf{AB}$ can be merged into $\mathbf{W}$ for zero inference overhead
- Used in virtually every production fine-tuning workflow: HuggingFace PEFT, Axolotl, LLaMA-Factory

LoRA works because the learned $\Delta\mathbf{W}$ for a fine-tuning task is empirically low-rank. The intrinsic dimensionality of adaptation is much smaller than d^2_{model}.

GenAI Application: Multimodal Alignment Projections

Bridging Vision and Language Spaces

- Vision encoders (CLIP, ViT) output image embeddings in a different space than text embeddings
- A **projection layer** (MLP or linear) maps vision embeddings into the LLM's d_{model}-dimensional space
- **LLaVA (2023):** Single linear projection $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{d_{\text{vision}} \times d_{\text{model}}}$ trained on image-text pairs
- **LLaVA-1.5:** Two-layer MLP projection (two projection layers) for richer alignment
- **GPT-4V, Gemini:** Proprietary projection architectures; conceptually the same

GenAI Application: Adapter Layers

Bottleneck Adapters

- Adapter layers insert a **down-project, nonlinearity, up-project** bottleneck inside the Transformer
- Down: $d_{\text{model}} \to r$ (small rank)
- Up: $r \to d_{\text{model}}$
- Only adapter weights trained; base frozen
- Used in AdapterHub, Google's T5 adapter work

Versus LoRA

- Both exploit low-rank structure
- Adapters add *sequential* depth; LoRA adds *parallel* branches
- LoRA can be merged at inference; adapters cannot
- LoRA has become dominant in production fine-tuning

GenAI Application: Quantizing Projection Layers

Weight-Only Quantization Targets Projections

- GPTQ and AWQ quantize projection weight matrices to 4-bit or 8-bit integers
- Because projections are purely linear, quantization error is relatively bounded and predictable
- A 4-bit quantized Llama 3 8B fits in $\approx$4 GB VRAM (vs $\approx$16 GB in FP16)
- Activation projections (Q, K, V, O) are most sensitive to quantization; FFN projections are more robust
- bitsandbytes library implements NF4 quantization of projection layers used in QLoRA

Because projection layers dominate parameter count and memory bandwidth, quantizing them is the single most effective way to reduce LLM deployment cost. Modern 4-bit quantized models lose $<$1% on standard benchmarks.

Common Misconceptions

Myths and Corrections

- **Myth:** "Projection layers include activation functions." Standard projections are purely linear; activations belong to the next stage (e.g., GELU after $\mathbf{W}_1$ in FFN)
- **Myth:** "LoRA is only for attention." LoRA can be applied to FFN projection matrices too; some configurations include all six projection types per layer
- **Myth:** "Larger d_{ff} always improves quality." The optimal ratio $d_{\text{ff}}/d_{\text{model}}$ depends on data size; wider FFNs overfit on small datasets
- **Myth:** "All attention heads use the same projection matrices." Each head has its own $\mathbf{W}_i^Q$, $\mathbf{W}_i^K$, $\mathbf{W}_i^V$; only $\mathbf{W}^O$ is shared

Key Takeaways

What You Have Learned

- A projection layer is a single learned linear map $\mathbf{y} = \mathbf{x}\mathbf{W}$; no activation function
- Each Transformer block contains six projection matrices: Q, K, V, O (attention), and up/down (FFN)
- Q, K, V projections create different "views" of the same input; $\mathbf{W}^O$ merges all heads
- FFN projections follow an expand-activate-compress pattern; SwiGLU adds a third gate matrix in modern models
- LoRA exploits the low-rank structure of projection weight updates for parameter-efficient fine-tuning
- Multimodal projections bridge vision and language spaces; quantizing projections enables efficient deployment

Thank You

Projection Layers — Module 3.4